RAG Chatbot — Notebook Explanations

Insurance Policy Q&A Chatbot
Author: Suresh D R | AI Product Developer & Technology Mentor

---

Part 1 — AWS Setup Before Running Any Notebook

Do all of these steps once before opening any Colab notebook. Once done, you never need to repeat them.

Step 1 — Create an IAM User (Do NOT use Root Account)

The root account is your main AWS account. Never use it directly for development — one mistake can delete everything. Create a separate IAM user for this project.

Go to: AWS Console → Search "IAM" → Click IAM → Left sidebar: Users → Create User

1. Username: `rag-developer`
2. Do NOT check "provide console access" — we only need programmatic access
3. Click Next → Select "Attach policies directly"
4. Search and attach: **AdministratorAccess** (covers all services needed across all projects)
5. Click Next → Create User

> **Why AdministratorAccess?** This is a personal learning account. You need S3, ECS, ECR, Lambda, SageMaker, Bedrock across multiple projects. Adding policies one by one wastes time. For a company account, you would restrict permissions.

Step 2 — Create Access Keys

After the user is created, you need keys so Colab can connect to AWS.

Go to: IAM → Users → Click `rag-developer` → Security credentials tab → Scroll to Access keys → Create access key

1. Use case: Select **Command Line Interface (CLI)**
2. Click Next → Create access key
3. You will see two values:
   - **Access Key ID** — looks like `AKIAIOSFODNN7EXAMPLE`
   - **Secret Access Key** — looks like `wJalrXUtnFEMI/K7MDENG/bPxRfiCY`
4. **Download the CSV immediately** — the secret key cannot be seen again after you close this page

> **Important:** Never share these keys publicly, never paste them in GitHub, never upload them in a notebook to GitHub. Store them only in your downloaded CSV and in Colab Secrets or paste directly in the notebook credentials cell.

Step 3 — Enable MFA on Root Account (2 minutes, strongly recommended)

Go to: AWS Console → Top right: your account name → Security credentials → Multi-factor authentication → Assign MFA device → Follow the steps with your phone's authenticator app.

This protects your account if the root credentials are ever leaked.

Step 4 — Create the S3 Bucket

Go to: AWS Console → Search "S3" → Open S3 → Create bucket

Setting	Value
Bucket name	`insurance-rag-bucket-2026` (or any unique name)
Region	**eu-north-1** (Stockholm) or **ap-south-1** (Mumbai) — pick one and use it everywhere
Block all public access	**ON** (leave default — we do not want public access)
Versioning	**Enable** — useful when you update policy documents
Everything else	Leave as default

Click **Create bucket**.

Step 5 — Create the Folder Structure Inside the Bucket

Open the bucket → Click **Create folder** and create these folders one by one.

Folders to create:

```
insurance-rag-bucket-2026/
│
├── raw/                          ← Upload your original files here
│   ├── star-health/              ← Word documents go here
│   └── data/                     ← CSV files go here
│
├── processed/                    ← Created automatically by Notebook 01
│   (Notebook 01 saves chunks, ChromaDB backup, quality report here)
│
├── indexes/                      ← Created automatically by Notebook 02
│   (Notebook 02 saves BM25 index here)
│
└── prompts/                      ← Created automatically by Notebook 03
    (Notebook 03 saves all prompt templates here)
```

You only need to manually create these 3 folders:

- `raw/`
- `raw/star-health/`
- `raw/data/`

The `processed/`, `indexes/`, and `prompts/` folders are created automatically when the notebooks run and save files.

Step 6 — Upload Your Dataset Files

Open the bucket → click into each folder → click Upload → Add files → Upload.

File	Upload to folder
`star_health_comprehensive_policy.docx`	`raw/star-health/`
`claims_guidelines.docx`	`raw/star-health/`
`agent_product_manual.docx`	`raw/star-health/`
`hospital_network.csv`	`raw/data/`
`product_comparison.csv`	`raw/data/`
`customer_qa_history.csv`	`raw/data/`

After uploading, your bucket should look like this:

```
raw/star-health/star_health_comprehensive_policy.docx
raw/star-health/claims_guidelines.docx
raw/star-health/agent_product_manual.docx
raw/data/hospital_network.csv
raw/data/product_comparison.csv
raw/data/customer_qa_history.csv
```

Step 7 — Get Your OpenAI API Key

Go to: platform.openai.com → Sign in → Top right: your profile → API keys → Create new secret key

Copy the key — it starts with `sk-`. You will paste this into the credentials cell in each notebook.

> **Cost estimate:** Running all 4 notebooks completely will cost approximately $1 to $3 in OpenAI API calls (embeddings + GPT-4o calls). The RAGAS evaluation step costs the most.

Step 8 — What Goes Into the Notebooks

In Step 2 of every notebook, paste your credentials here:

```
OPENAI_API_KEY        = "sk-your-openai-key-here"
AWS_ACCESS_KEY_ID     = "your-access-key-id"
AWS_SECRET_ACCESS_KEY = "your-secret-access-key"
AWS_REGION            = "eu-north-1"        # same region you chose for your bucket
S3_BUCKET             = "insurance-rag-bucket-2026"   # your bucket name
```

This is the only place you change credentials. Every other cell reads from these variables.

What S3 Looks Like After All 4 Notebooks Run

Once you have run all 4 notebooks end to end, your S3 bucket will contain:

```
insurance-rag-bucket-2026/
│
├── raw/
│   ├── star-health/
│   │   ├── star_health_comprehensive_policy.docx    ← uploaded by you
│   │   ├── claims_guidelines.docx                   ← uploaded by you
│   │   └── agent_product_manual.docx                ← uploaded by you
│   └── data/
│       ├── hospital_network.csv                     ← uploaded by you
│       ├── product_comparison.csv                   ← uploaded by you
│       └── customer_qa_history.csv                  ← uploaded by you
│
├── processed/                                       ← created by Notebook 01
│   ├── all_chunks.json                              ← all text chunks with metadata
│   ├── chromadb_backup.zip                          ← vector database backup
│   ├── chunk_quality_report.json                    ← chunk quality scores
│   ├── golden_dataset.json                          ← evaluation Q&A pairs
│   ├── ragas_eval_results.json                      ← RAGAS scores          (NB04)
│   └── final_pipeline_results.json                  ← test run results       (NB04)
│
```

```
├── indexes/                                    ← created by Notebook 02
│   └── bm25_index.pkl                          ← BM25 keyword search index
│
└── prompts/                                    ← created by Notebook 03
    ├── system_prompt.txt
    ├── factual_qa.txt
    ├── calculation.txt
    ├── summarisation.txt
    └── hallucination_check.txt
```

AWS Services Used in This Project

Service	What It Does in This Project	When It Is Used
S3	Stores all raw files, processed chunks, ChromaDB backup, BM25 index, prompts	All notebooks
IAM	Manages access — your `rag-developer` user and its access keys	Setup only
ECS Fargate	Runs the production FastAPI and Celery containers	Production phase (after notebooks)
ECR	Stores Docker images for production deployment	Production phase
Lambda	Event trigger when new documents are uploaded to S3	Production phase
SQS	Queue for async document processing jobs	Production phase
Secrets Manager	Stores API keys securely in production (replaces hardcoded keys)	Production phase
CloudWatch	Monitors latency, errors, hallucination rate in production	Production phase
Bedrock	Alternative to OpenAI — Claude or Titan hosted on AWS (data stays in AWS)	Optional

> For the notebook experimentation phase, only **S3 and IAM** are needed. Everything else is for the production deployment in Option B.

Part 2 — Quick Concept Reference

Before reading the notebooks, here are the key technical terms explained simply:

RAG (Retrieval Augmented Generation)
Instead of answering from memory, the AI first searches your documents, finds the relevant paragraphs, and then writes the answer based only on what it found. Like an open-book exam — the answer comes from the book, not from memory.

Embeddings
A way to convert text into a list of numbers that captures its meaning. Two sentences that mean similar things will have similar numbers. This is what makes semantic search possible.

Vector Database (ChromaDB)
A special database that stores embeddings and can quickly find the most similar ones to a query. We use ChromaDB — it runs directly inside Colab with zero setup.

BM25
A keyword-based search algorithm. It finds documents that contain the exact words in your query. Works well for specific terms like policy numbers, drug names, or clause references that vector search might miss.

Hybrid Search
Combines vector search (finds meaning) and BM25 (finds exact words) into one result list. Better than either method alone.

Reciprocal Rank Fusion (RRF)
The algorithm that merges vector and BM25 results. A chunk that appears high in both lists gets a higher combined score. A chunk that only appears in one list gets a lower score.

Cross-Encoder Re-Ranking
After hybrid search retrieves the top 20 chunks, a cross-encoder model reads the question AND each chunk together and re-scores them. Unlike similarity search which compares separately, the cross-encoder sees both at the same time — so it gives a much more accurate relevance score. Think of it as a second, smarter opinion on which chunks are truly the most relevant.

Hallucination
When an AI generates information that was not in the retrieved documents. For example, the policy says room rent is Rs.3,000 per day but the AI says Rs.5,000. Hallucination detection catches this before showing the answer to the customer.

RAGAS
A framework for evaluating RAG systems. Measures 4 things — Faithfulness (is the answer grounded in the documents?), Answer Relevance (does the

answer address the question?), Context Precision (are the retrieved chunks useful?), Context Recall (did we retrieve everything needed?).

Guardrails
Safety checks that run before and after the AI generates an answer. Input guardrails block harmful questions (PII, prompt injection, out-of-scope). Output guardrails verify citations and add mandatory disclaimers.

Part 3 — How the 4 Notebooks Connect

```
Notebook 01
Downloads data from S3 → Parses documents → Chunks → Stores in ChromaDB → Saves to S3
        ↓
Notebook 02
Loads ChromaDB from S3 → Builds retrieval system → Hybrid search → Re-ranking → Saves BM25 to S3
        ↓
Notebook 03
Loads ChromaDB + BM25 from S3 → Adds intelligence layer → Classification → Prompts → Guardrails → Full pipeline
        ↓
Notebook 04
Loads everything from S3 → Evaluates quality → Confidence scoring → Final chatbot
```

Each notebook saves its outputs to S3. The next notebook loads from S3. Nothing is lost if Colab restarts.

Part 4 — Notebook 01 — Data Ingestion, Parsing and Chunking

What this notebook is about:
Getting raw data from S3, converting it into clean searchable chunks, and storing those chunks in ChromaDB.

Step by step:

Step 1 — Install libraries
Installs boto3 (AWS connection), python-docx (Word file parsing), ChromaDB (vector database), OpenAI (embeddings), and LangChain (chunking utilities).

Step 2 — Set credentials
One place to paste your OpenAI API key and AWS access keys. All notebooks use the same keys.

Step 3 — Connect to S3
Connects to your AWS S3 bucket and lists all files to verify the connection is working. You should see the 6 dataset files you uploaded.

Step 4 — Download files
Downloads all 6 files from S3 to Colab's local storage — 3 Word documents (policy, claims guidelines, agent manual) and 3 CSV files (hospital network, product comparison, Q&A history).

Step 5 — Parse Word documents
Opens each Word document and extracts all text paragraph by paragraph, tracking which section each paragraph belongs to. Tables are extracted separately because they contain critical data like sub-limits and benefit amounts.

Step 6 — Parse CSV files
Loads all 3 CSV files into Pandas DataFrames and prints a summary — shape, column names, sample rows, and key statistics for each file.

Step 7 — Clean text
Removes noise from extracted text — extra whitespace, standalone page numbers, repeated headers and footers. Clean text leads to better embeddings.

Step 8 — Detect section type and chunk
Identifies what type of content each section contains (coverage, exclusion, definition, claims, table etc.) and applies the right chunking strategy:

- Tables → always kept as one chunk, never split
- Exclusions and definitions → small chunks (400 chars) — each clause is one complete idea
- Coverage and general text → medium chunks (700 chars) with overlap so no sentence is cut in the middle

Step 9 — Convert CSV rows to text chunks
Converts each CSV row into a readable sentence so it can be embedded and searched like a document. Each hospital row becomes one descriptive sentence. Each product comparison row becomes one sentence with all plan features.

Step 10 — Chunk quality evaluation
Before indexing, checks 4 things:

- Size distribution — flags chunks that are too tiny (under 80 chars, likely parsing artifacts) or too large (over 2000 chars, likely multiple topics mixed)
- Section coverage — verifies all expected types (coverage, exclusion, definition, claims, table) are present

- Self-containment — GPT-4o-mini reads 15 random chunks and checks if each one makes sense on its own without needing surrounding context
- Duplicate detection — removes identical chunks

Step 11 — Embed and store in ChromaDB
Sends all chunks to OpenAI's text-embedding-3-large model to generate vector representations. Stores vectors plus metadata (chunk type, section name, plan name, insurer) in ChromaDB. Processes in batches of 50 to respect rate limits.

Step 12 — Save to S3
Saves three things back to S3:

- all_chunks.json — all chunks with metadata (used by Notebook 02 for BM25)
- chromadb_backup.zip — the entire ChromaDB database (loaded by Notebooks 02, 03, 04)
- chunk_quality_report.json — quality scores and issue summary

Step 13 — Quick search test
Runs 5 sample questions through ChromaDB to verify retrieval is working. Checks that similarity scores are reasonable and chunk types match the question topics.

Step 14 — Interactive widget
Type any question to see which chunks ChromaDB retrieves. Use this to verify the right section types are being returned before moving to Notebook 02.

What good output looks like:

- Total chunks created: 150 to 300
- Quality score above 0.75
- Self-containment score above 0.75
- Search test shows similarity scores above 0.50

Part 5 — Notebook 02 — Hybrid Search and Re-Ranking

What this notebook is about:
Building a better retrieval system. Vector search alone is not enough — this notebook adds BM25 keyword search and cross-encoder re-ranking, then measures which combination is best.

Step by step:

Step 1 — Install libraries
Installs rank-bm25 (keyword search), sentence-transformers (cross-encoder model).

Step 2 — Set credentials
Same as Notebook 01.

Step 3 — Load ChromaDB from S3
Downloads the ChromaDB backup saved by Notebook 01, extracts it, and reloads the collection. Also loads all_chunks.json which is needed to build the BM25 index.

Step 4 — Vector search
Embeds the question with OpenAI and searches ChromaDB for the most similar chunks by cosine similarity. Returns top 20 results. Good at finding semantically related content even when the customer uses different words.

Step 5 — BM25 keyword search
Builds a BM25 index on all chunk texts. BM25 finds exact keyword and phrase matches — useful for specific terms like policy section numbers, drug names, or exact clause references that vector search might rank lower. The BM25 index is saved to S3 for reuse.

Step 6 — Hybrid search with RRF
Combines vector results and BM25 results using Reciprocal Rank Fusion. The formula is simple — score = 1/(k + rank) for each list, summed. A chunk that ranks high in both lists gets a much higher combined score. A chunk that only appears in one list scores lower. This improves context recall by 20 to 30% over vector search alone.

Step 7 — Cross-encoder re-ranking
Loads the cross-encoder model (ms-marco-MiniLM-L-6-v2). After hybrid search retrieves 20 chunks, the cross-encoder reads the question and each chunk together and gives each a relevance score from -10 to +10. The top 5 after re-ranking are much more precisely relevant than the top 5 from initial retrieval.

Step 8 — Quality comparison
Measures context precision of all 3 methods on 5 test questions. For each question and each method, GPT-4o-mini judges whether each retrieved chunk actually helps answer the question. Shows clearly that Hybrid + Rerank outperforms the others.

Step 9 — Interactive widget
Dropdown to pick retrieval method (Vector Only, BM25 Only, Hybrid, Hybrid+Rerank) and ask any question. Compare how each method finds different chunks. Useful for understanding why the hybrid approach is better.

What good output looks like:

- Vector precision: around 0.60 to 0.70
- Hybrid precision: around 0.70 to 0.80
- Hybrid + Rerank precision: around 0.80 to 0.90
- Rerank scores above 5.0 for the top chunk

Part 6 — Notebook 03 — Query Classification, Prompts and Guardrails

What this notebook is about:
Adding intelligence to the retrieval pipeline. Not all questions are the same — a comparison question needs a different strategy than a calculation question. This notebook classifies questions and routes them correctly, then builds all the safety and quality checks.

Step by step:

Step 1 — Install libraries
Installs Presidio (PII detection) and spaCy (named entity recognition used by Presidio).

Step 2 — Set credentials and load everything
Loads ChromaDB, all chunks, BM25 index, and prompt files from S3. Also initialises the PII analyzer and cross-encoder reranker. Everything saved by Notebooks 01 and 02 is loaded here.

Step 3 — Reload retrieval functions
Redefines all retrieval functions from Notebook 02 — vector search, BM25 search, hybrid search with RRF, and cross-encoder reranking. These are used throughout this notebook.

Step 4 — Query classifier
Sends the customer question to GPT-4o-mini and classifies it into one of 8 types:

- factual — simple fact lookup
- comparison — comparing two plans
- summarisation — asking for a summary
- multi_hop — needs multiple facts combined

- eligibility — checking if something qualifies
- claims_process — about filing or tracking claims
- calculation — needs arithmetic
- negation — asking what is NOT covered

Each type routes to a different retrieval strategy. Tested on 8 sample questions with accuracy score.

Step 5 — Query decomposition
For multi-hop questions that need multiple facts combined, the question is broken into 2 to 4 simple sub-questions. Each sub-question is retrieved separately. The answers are combined into one final answer. Example: "I have BP, 14 months into policy, need angioplasty — covered?" becomes 3 separate retrievals.

Step 6 — RAG Fusion
For ambiguous questions like "Is my surgery covered?", generates 4 different ways to ask the same question — one targeting coverage, one exclusions, one limits, one eligibility. Retrieves for all 4 variants and merges results with RRF. Gets much better coverage of the topic.

Step 7 — Prompt templates
Builds 5 prompt templates and saves them to S3:

- System prompt — defines the AI's role and rules (always cite source, never guess, use simple language)
- Factual Q&A prompt — for most question types
- Calculation prompt — instructs the AI to show working in a table format
- Summarisation prompt — instructs structured summary by category
- Hallucination check prompt — used to verify answers are grounded

Step 8 — Input guardrails
Checks every incoming question before processing:

- PII detection using Presidio — blocks Aadhaar numbers, phone numbers, email addresses
- Prompt injection detection — blocks "ignore instructions", "act as", "forget your rules" etc.
- Out of scope detection — blocks weather, coding, non-insurance questions
- Length limit — blocks questions over 2000 characters

Tested on 8 cases showing PASS or BLOCK with the reason.

Step 9 — Hallucination detection
After generating an answer, a second GPT-4o-mini call reads the retrieved chunks and the generated answer. It checks whether every factual claim in the answer is actually supported by the chunks. Returns SUPPORTED or UNSUPPORTED. If UNSUPPORTED, the confidence score is penalised.

Step 10 — Full pipeline function
Wires all components into one `rag_pipeline()` function:

1. Input guardrails
2. Query classification
3. Retrieve using the right strategy for the query type
4. Cross-encoder reranking
5. Build the right prompt template
6. Generate answer with GPT-4o
7. Hallucination check
8. Output guardrails and disclaimer
9. Generate 3 suggested follow-up questions

Tested on 5 questions covering different query types.

Step 11 — Interactive widget
Full pipeline chatbot — type any question and get the complete answer with query type detected, sources used, grounding status, and 3 suggested follow-up questions.

What good output looks like:

- Classifier accuracy: 7/8 or 8/8 on test questions
- Guardrails blocking all 3 harmful test cases and passing all normal ones
- Hallucination verdict: SUPPORTED for well-grounded answers
- Full pipeline answers include a source section reference

Part 7 — Notebook 04 — Confidence Scoring, Evaluation and Final Chatbot

What this notebook is about:
Measuring how good the system is and building the final chatbot. Every answer gets a confidence score. RAGAS evaluation tells us exactly where the system is strong and where it needs improvement.

Step by step:

Step 1 — Install RAGAS and evaluation libraries
Installs RAGAS (evaluation framework) and Hugging Face datasets.

Step 2 — Set credentials and load everything
Loads ChromaDB, chunks, BM25 index, all prompts, reranker, and PII analyzer from S3. Everything from all previous notebooks is loaded here.

Step 3 — Reload all pipeline functions
All functions from Notebooks 02 and 03 are redefined — retrieval, reranking, classification, decomposition, RAG Fusion, guardrails, and hallucination detection.

Step 4 — Confidence scorer
Combines 4 signals into a single score from 0 to 1:

- Retrieval similarity average — how similar were the top chunks to the question? (40% weight)
- Cross-encoder reranking score — how relevant was the best chunk? (40% weight)
- Hallucination penalty — -0.30 if the answer was not fully grounded
- Query type modifier — multi-hop and calculation questions get a small penalty as they are inherently harder

Score categories:

- 0.75 and above → HIGH — show answer normally
- 0.50 to 0.74 → MEDIUM — suggest verifying for important decisions
- Below 0.50 → LOW — show warning, flag for human review

Step 5 — Golden dataset and RAGAS evaluation
Loads the customer Q&A history CSV from S3. Filters to resolved questions to use as the golden dataset. Runs the retrieval and generation pipeline on 10 questions, then measures 4 RAGAS metrics:

- Faithfulness — is the answer only using information from the retrieved chunks? (target above 0.85)
- Answer Relevance — does the answer actually address the question asked? (target above 0.80)

- Context Precision — of the chunks retrieved, how many were actually useful? (target above 0.75)
- Context Recall — did we retrieve all the information needed to answer correctly? (target above 0.80)

Step 6 — Diagnose weak questions
Shows per-question scores and identifies questions where the overall score is below 0.65. For each weak question, diagnoses the root cause:

- Low faithfulness → system is hallucinating, strengthen the system prompt
- Low context recall → missing relevant chunks, increase retrieval top_k
- Low context precision → retrieving too much noise, tighten reranking
- Low answer relevance → wrong query type classified, review the classifier

Step 7 — Final complete pipeline
The final `final_rag_pipeline()` function with all 9 components wired together including confidence scoring. Tested on 5 questions. The output shows confidence score, category, latency, and review flag for each question.

Step 8 — Save results to S3
Saves RAGAS evaluation scores and final pipeline test results to S3 for reference and comparison after future improvements.

Step 9 — Final chatbot interface
A full chatbot UI with:

- Dropdown of 8 sample questions to try
- Free text input for any custom question
- Confidence badge — HIGH in green, MEDIUM in yellow, LOW in red
- Source citations showing which policy section each answer came from
- Hallucination status
- Review flag shown prominently if confidence is LOW
- 3 suggested follow-up questions after every answer

What good output looks like:

- RAGAS Faithfulness above 0.80
- RAGAS Answer Relevance above 0.75
- Overall RAGAS average above 0.78

- Confidence score HIGH for simple factual questions
- Final chatbot showing sources and suggestions correctly

Part 8 — Common Errors and Fixes

Error	Fix
`NoneType has no attribute name` in parse_docx	Add `if para.style and` before `para.style.name.startswith`
`No module named langchain.text_splitter`	Change to `from langchain_text_splitters import RecursiveCharacterTextSplitter`
`ValueError: Expected where to have exactly one operator` in ChromaDB	Use `{'$and': [{'tenant_id': 'star-health'}, {'chunk_type': ctype}]}` when filtering multiple fields
RAGAS ModuleNotFoundError langchain_community	Run `!pip install ragas==0.1.21 langchain-community langchain-openai -q` then restart runtime
Low similarity scores (below 0.50)	Normal for short questions — hybrid + rerank will fix this in NB02

Author: Suresh D R | AI Product Developer & Technology Mentor